

BVSP – Technical Implementation Guide (TIG)

Version 1.0

Project: Bhanap Vaiyaktika Seva Portal (BVSP)

Version: 1.0

Document Status: Approved

Prepared By: Dr. Vishweshwar Kulkarni Foundation

Technology Partner: V K Computer Services (VKCS)

1. Purpose

The Technical Implementation Guide (TIG) defines the software architecture, technology stack, coding standards, database design, UI standards, security principles, and development conventions for BVSP.

It is the primary technical reference for Google AI Studio, Firebase Studio, and all future developers.

2. Guiding Principles

- Production-ready code only.
 - Mobile-first and responsive.
 - Senior citizen friendly.
 - Configuration-driven architecture.
 - No hardcoded Sabha or Programme names.
 - Modular and reusable components.
 - Scalable for multiple Sabhas and Programmes.
 - Clean, maintainable TypeScript code.
 - Secure by default.
-

3. Technology Stack

Frontend

- React 18
- TypeScript
- Vite
- Tailwind CSS
- shadcn/ui
- Lucide Icons
- Framer Motion (minimal)

Backend

- Firebase Authentication
- Cloud Firestore
- Firebase Storage

Deployment

- Cloudflare Pages
- GitHub Repository

Future Ready

- Firebase Cloud Functions
- Firebase Messaging
- Firebase Analytics

4. Application Architecture

The application follows a layered architecture:

```
Presentation Layer
|
Business Logic Layer
|
Service Layer
|
Firebase
|
Firestore / Storage / Authentication
```

Business logic must never be placed directly inside UI components.

5. Folder Structure

```
src/
├── assets/
├── components/
├── config/
├── contexts/
├── firebase/
├── hooks/
├── layouts/
├── pages/
├── services/
├── styles/
├── types/
├── utils/
├── App.tsx
└── main.tsx
```

Each folder has a single responsibility.

6. Routing

Public

- /
- /about
- /programmes
- /programmes/:id
- /resources
- /faq
- /important-dates
- /login

Authenticated

- /dashboard
- /my-seva
- /daily-log
- /progress
- /summary

Coordinator

- /coordinator

Super Admin

- /admin

7. Firestore Collections

sabhas

users

programmes

elements

resources

sankalpas

dailyLogs

darshan

subhashitam

inspirations

importantDates

faq

settings

auditLogs

All collections must use document IDs rather than embedded hardcoded values.

8. Data Model Principles

Every document should include:

id

createdAt

updatedAt

createdBy

updatedBy

status

Use Firebase Timestamp.

Avoid duplicate data.

Reference related documents using IDs.

9. Authentication

Version 1 Authentication:

- Mobile Number
- 4-digit PIN

Authentication handled by Firebase Authentication.

Session management must persist securely.

No passwords.

No email login.

10. User Roles

Guest

Public access only.

Sadhaka

Personal dashboard.

Daily logging.

Progress.

Coordinator

Sabha management.

Reports.

Approvals.

Super Admin

Complete system administration.

11. UI Design Standards

Theme:

Primary

- Deep Teal

Secondary

- Temple Gold

Accent

- Saffron

Background

- Warm Ivory

Cards

Rounded corners

Soft shadows

Large touch targets

Readable typography

Minimal animations

12. Responsive Design

Support:

Desktop

Tablet

Mobile

Navigation adapts automatically.

Never create desktop-only pages.

13. Accessibility

Large typography.

High contrast.

Large buttons.

Keyboard navigation.

Screen reader friendly.

Minimal cognitive load.

14. Component Standards

Create reusable components.

Examples:

PageHeader

Card

SectionTitle

ProgrammeCard

ResourceCard

LoadingSpinner

ErrorCard

SearchBox

ConfirmationDialog

DataTable

Never duplicate UI code.

15. Service Layer

All Firestore access goes through services.

Example:

ProgrammeService

UserService

SankalpaService

ResourceService

AuthService

ReportService

Pages should never query Firestore directly.

16. Error Handling

Centralized.

User-friendly messages.

Graceful fallback.

Logging for debugging.

17. Performance

Lazy loading.

Code splitting.

Image optimization.

Firestore pagination.

Caching where appropriate.

18. Security

Role-based access.

Firestore Security Rules.

Validate all user input.

Prevent unauthorized document access.

Never expose secrets.

Environment variables only.

19. Coding Standards

Use TypeScript strict mode.

Meaningful variable names.

Small reusable functions.

Single responsibility principle.

Avoid nested logic.

Comment only when necessary.

20. Naming Conventions

Components:

`ProgrammeCard.tsx`

Hooks:

`useAuth.ts`

Services:

`ProgrammeService.ts`

Types:

`Programme.ts`

21. Git Workflow

Main Branch

Production-ready only.

Feature Branch

One feature per branch.

Commit Messages

`feat:`

`fix:`

`refactor:`

docs:

22. Testing

Every module should be verified for:

Responsive layout

Authentication

Firestore operations

Role permissions

Performance

Accessibility

23. Deployment

Development

GitHub

↓

Firebase Studio / Google AI Studio

↓

Review

↓

Cloudflare Pages

↓

Production

24. Future Scalability

Architecture must support:

Multiple Sabhas

Multiple Programmes

Multiple Languages

Push Notifications

Certificates

Donations

Offline Mode

Without architectural redesign.

25. AI Development Instructions

When generating code:

- Follow this Technical Implementation Guide.
 - Refer to the Business Requirements Blueprint for functional behavior.
 - Follow the current Sprint Build Specification.
 - Do not invent functionality beyond the specification.
 - Use TODO comments where requirements are intentionally deferred.
 - Generate production-quality, modular, and maintainable code.
-

Technical Freeze (Version 1.0)

The following architectural decisions are frozen for Version 1.0:

- **Frontend:** React + TypeScript + Vite
- **UI:** Tailwind CSS + shadcn/ui
- **Backend:** Firebase Authentication, Firestore, Storage
- **Hosting:** Cloudflare Pages

- **Authentication:** Mobile Number + 4-digit PIN
- **Architecture:** Configuration-driven, multi-Sabha and multi-programme ready
- **Database:** Firestore with service-layer access only
- **Design:** Mobile-first, senior citizen friendly, reusable component architecture

Appendix A – Firestore Schema & Data Dictionary

BVSP Technical Implementation Guide v1.0

Design Principles

- Every collection is independent.
 - Documents are connected using IDs (references), not duplicated data.
 - Every document includes audit fields.
 - No Sabha names or Programme names are hardcoded.
 - New Sabhas and Programmes can be added without changing the application code.
-

Standard Audit Fields (All Collections)

Every document must contain:

Field	Type	Description
id	String	Firestore Document ID
status	String	Active / Inactive / Archived
createdAt	Timestamp	Record creation time
createdBy	String	User ID
updatedAt	Timestamp	Last update time
updatedBy	String	User ID

1. sabhas

Stores participating Sabhas.

Field	Type
sabhaId	String
sabhaName	String
city	String

Field	Type
state	String
country	String
coordinatorId	String
logoURL	String
isActive	Boolean

Example

```
{
  "sabhaId": "GOREGAON",
  "sabhaName": "Goregaon Sabha",
  "city": "Mumbai",
  "isActive": true
}
```

2. users

Field	Type
userId	String
fullName	String
mobile	String
email	String
sabhaId	String
role	Guest / Sadhaka / Coordinator / SuperAdmin
pinHash	String
photoURL	String
isApproved	Boolean
lastLogin	Timestamp

3. programmes

Represents any spiritual programme.

Examples

- Chaturmas
- Parijnanlahari Mahotsava
- Guru Purnima
- Future Programmes

Field	Type
programmeId	String
programmeName	String
description	String
startDate	Date
endDate	Date
registrationRequired	Boolean
status	Active / Completed

4. elements

Every programme consists of one or more spiritual elements.

Examples

- Daily Japa
- Guru Smaran
- Reading
- Chanting
- Meditation

Field	Type
elementId	String
programmeId	String
elementName	String
description	String
displayOrder	Number

5. resources

Each resource belongs to an element.

Field	Type
resourceId	String
programmeId	String
elementId	String
title	String
description	String

Field	Type
resourceType	PDF / Audio / Video / Link
url	String
language	String
courtesy	String
displayOrder	Number

6. sankalpas

One Sankalpa per user per programme.

Field	Type
sankalpaId	String
userId	String
programmeId	String
date	Date
remarks	String

7. dailyLogs

Stores daily Sadhana entries.

Field	Type
logId	String
userId	String
programmeId	String
elementId	String
logDate	Date
completed	Boolean
remarks	String

8. darshan

Daily Darshan content.

Field	Type
darshanId	String
title	String
imageURL	String
description	String
displayDate	Date

9. subhashitam

Daily inspirational quotation.

Field	Type
subhashitamId	String
quote	String
source	String
language	String
displayDate	Date

10. inspirations

Daily inspiration or message.

Field	Type
inspirationId	String
title	String
content	String
imageURL	String
displayDate	Date

11. importantDates

Festival calendar and important events.

Field	Type
eventId	String

Field	Type
title	String
eventDate	Date
description	String
category	Festival / Sabha / Programme

12. faq

Frequently Asked Questions.

Field	Type
faqId	String
question	String
answer	String
category	String
displayOrder	Number

13. settings

Application configuration.

Field	Type
settingKey	String
value	Any
description	String

Examples

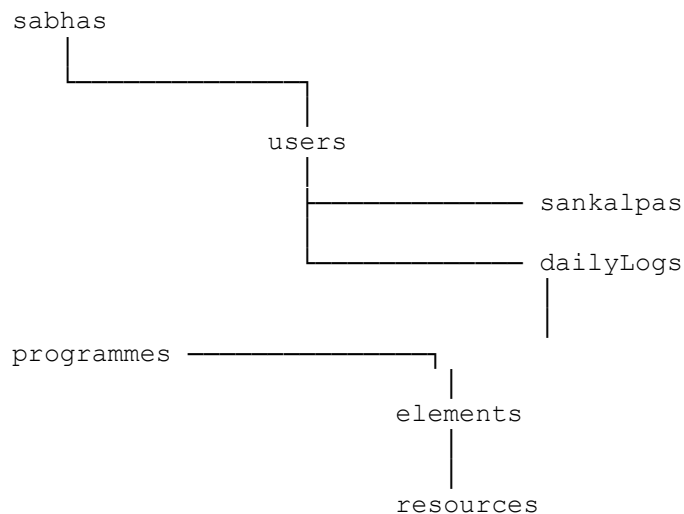
- Default Sabha
 - Theme
 - Contact Details
 - Support Email
 - Registration Open
 - Current Programme
-

14. auditLogs

System activity log.

Field	Type
logId	String
userId	String
action	String
module	String
timestamp	Timestamp
ipAddress	String
remarks	String

Collection Relationships



Data Integrity Rules

- Every user belongs to one Sabha.
- Every programme can have multiple elements.
- Every element can have multiple resources.
- A Sadhaka can submit only one Sankalpa per programme.
- Daily logs are linked to a programme and an element.
- Coordinators can access only users belonging to their Sabha.
- Super Admin has unrestricted access.
- Public content (Darshan, Subhashitam, Inspiration, FAQ, Important Dates) is readable without login.